

GRAFI

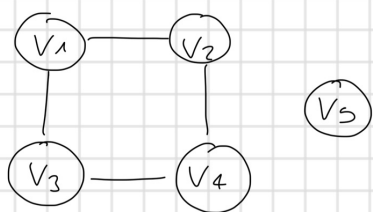
- Un grafo $G = (V, E)$ dove $V = \{v_1, v_2, \dots, v_n\}$ è l'insieme dei **vertici** mentre E è l'insieme degli **spigoli o archi**, $E \subseteq V \times V$, $e = (v_i, v_j) \in E$
- Un grafo è **NON ORIENTATO** se la relazione E è simmetrica, ossia $(v_i, v_j) \in E \Leftrightarrow (v_j, v_i) \in E$
- Un grafo è **ORIENTATO** se $(v_i, v_j) \in E$ non implica che $(v_j, v_i) \in E$

$G = (V, E)$ $|V| = n$ numero dei vertici
 $|E| = m$ numero degli spigoli

se $e = (u, v) \in E$ si dice che è incidente su u e v

- in un grafo non orientato il **grado di un vertice** è il numero di spigoli incidenti su di esso.
- in un grafo orientato si ha il **grado entrante** e **grado uscente**
- un vertice di grado 0 è **ISOLATO**
- un vertice connesso a tutti gli altri vertici del grafo è **UNIVERSALE**

G1

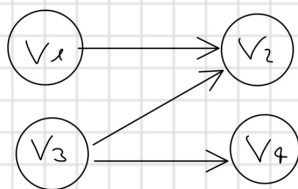


GRAFO
NON
ORIENTATO

v_5 ISOLATO

v_2 ha grado 2

G2

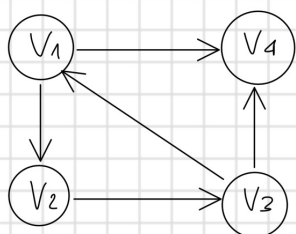


GRAFO ORIENTATO

v_1 ha grado uscente 2

v_1 ha grado entrante 0

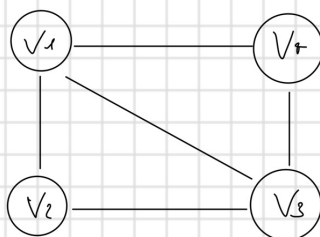
- Un **CAMMINO** di lunghezza k da vertice j_0 a j_{k+1} in G è una sequenza di vertici $v_0, v_1, v_2, \dots, v_k$ t.c. $\forall (v_i, v_{i+1}) \in E$
- Se $\exists$ un cammino da u a j si dice che j è raggiungibile da u
- Un ciclo è un cammino $v_1, v_2, \dots, v_k$ dove $v_1 = v_k$
- Un ciclo di lunghezza 1 è un **CAPPIO**
- Un grafo privo di cicli si dice **ACICLICO**
- Un grafo non orientato si dice **CONNESSO** se ogni coppia di vertici è collegata da un cammino.
- Un grafo orientato è **FORTEMENTE CONNESSO** se ogni coppia di vertici è collegata da un cammino.



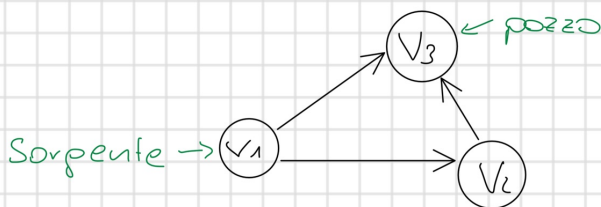
NON È FORTEMENTE CONNESSO



DEBOLMENTE CONNESSO

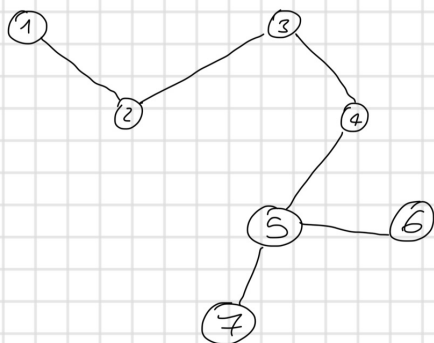


È CONNESSO

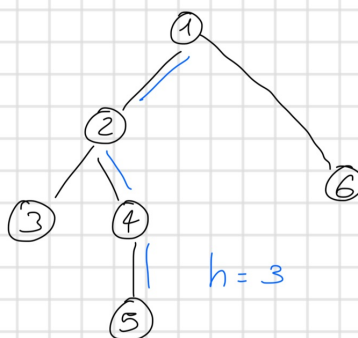


ALBERE

- Un **ALBERO** è un grafo non orientato, connesso e aciclico
- Un grafo non orientato è aciclico ma non connesso è una **FORESTA**
- Un **ALBERO RADICATO** è un albero in cui i nodi si distinguono come radice
- Se u è un vertice di un albero con radice r , ogni vertice v sull'unico cammino da r a u è **ANTENATO** di u mentre u è un **DISCENDENTE** di v
- Se (u, v) è uno spigolo dell'albero, u è **PADRE** e v è **FIGLIO**
- Un vertice senza figli è **FOGLIA**
- La lunghezza del cammino più lungo dalla radice ad una foglia è **ALTEZZA** dell'albero.



ALBERO LIBERO



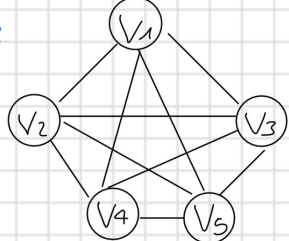
ALBERO RADICATO

Grafi - Strutture dati

- I grafi nella memoria di un computer possono essere rappresentati con:
 - **MATRICE DI ADIACENZA**: una matrice quadrata $n \times n$ se $|V| = n$. È binaria, ossia contiene 1 o 0; se $(i, j) \in E$, nella locazione (i, j) si mette 1, altrimenti 0.
 - **LISTE DI ADIACENZA**: per ogni vertice u si costruisce una lista di vertici adiacenti ad u .

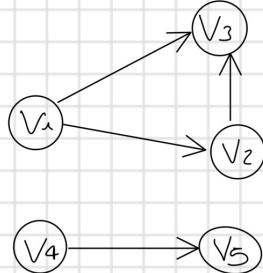
nel caso di grafi sparsi (con pochi spigoli rispetto al numero di vertici) sono convenienti le liste di adiacenza.

G_1



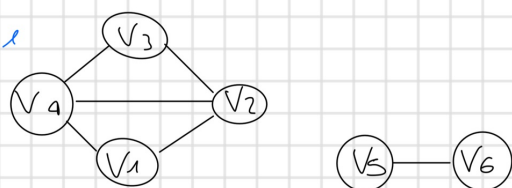
	1	2	3	4	5
1	0	1	1	1	1
2	1	0	1	1	1
3	1	1	0	1	1
4	1	1	1	0	1
5	1	1	1	1	0

G_2

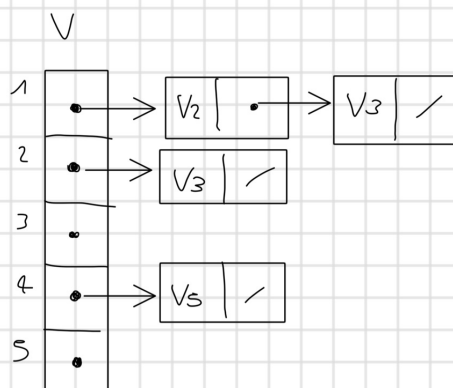
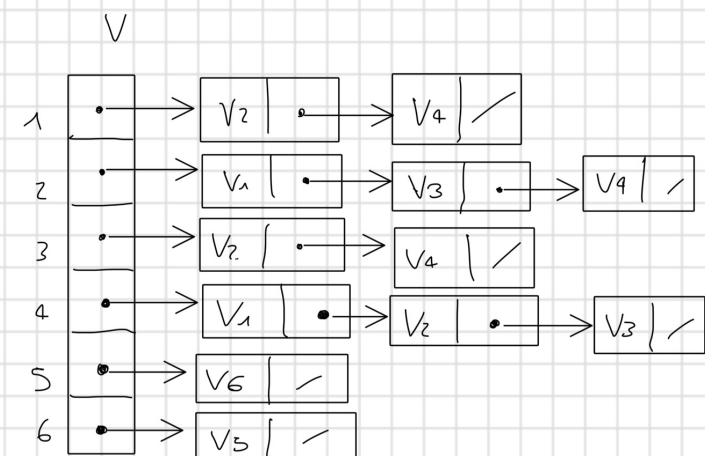
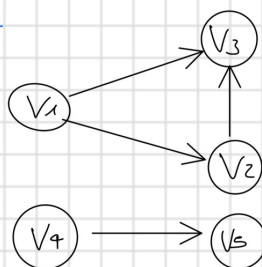


	1	2	3	4	5
1	0	1	1	0	0
2	0	0	1	0	0
3	0	0	0	0	0
4	0	0	0	0	1
5	0	0	0	0	0

G_1



G_2

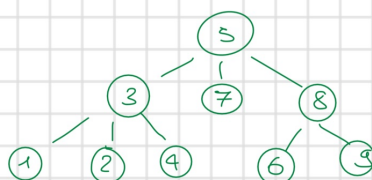
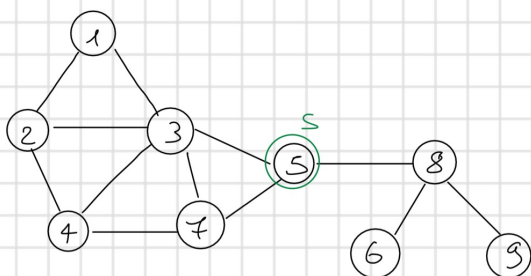


VISITA IN AMPIEZZA

Considerando una sorgente S :

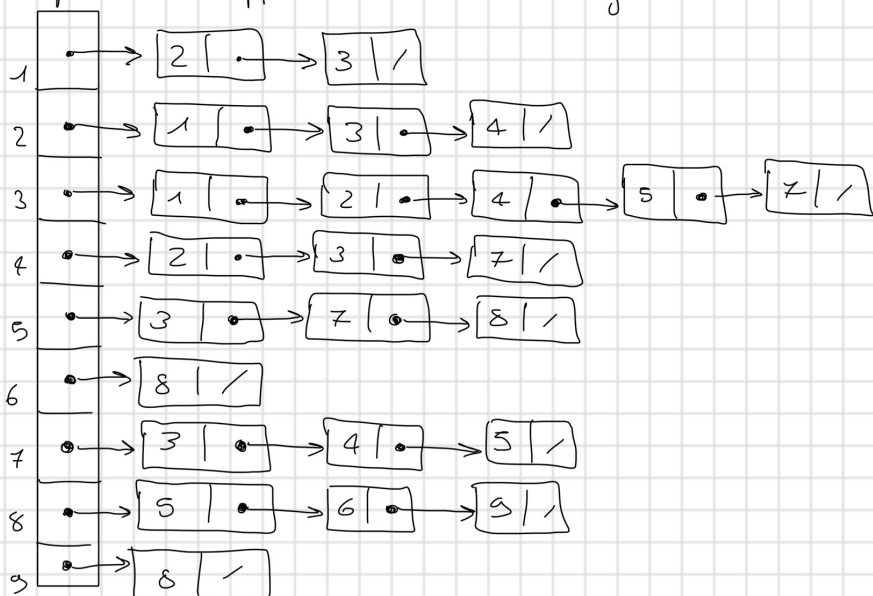
- visita i nodi a distanza crescente da S
- genera un albero (Breadth First) contenente tutti i vertici raggiungibili da S e tali che il cammino da S ad un nodo dell'albero corrispondono al cammino più breve nel grafo.
- calcola la distanza minima da S a tutti i nodi raggiungibili da esso.

Es:

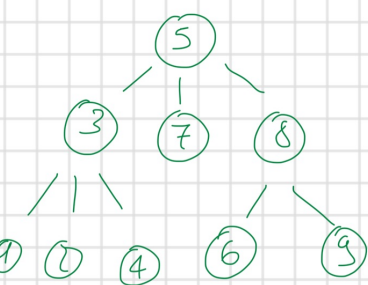
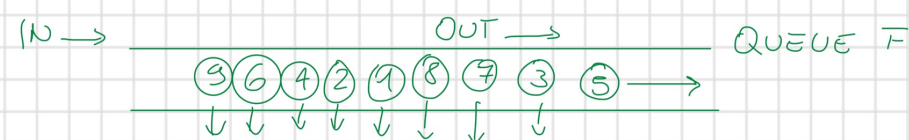


Per la visita in ampiezza si rappresenta con:

- per la rappresentazione del grafo usiamo la lista di adiacenza



	1	2	3	4	5	6	7	8	9
DIST	2	2	1	2	0	2	1	1	2
PARENT	3	3	5	3	/	8	5	5	8



PSEUDOCODICE

BFS (G, s)

for each v in V → inizializzo i valori
 PARENT[v] = null
 DIST[v] = -1

QUEUE F

DIST[s] = 0

F. ENQUEUE(s)

while (not F in empty) ← finché la coda non si svuota

$u \leftarrow F.$ DEQUEUE ← scoda u

for each v adiacente a u ← scorri lista adiacenza di u

if (DIST[v] == -1)

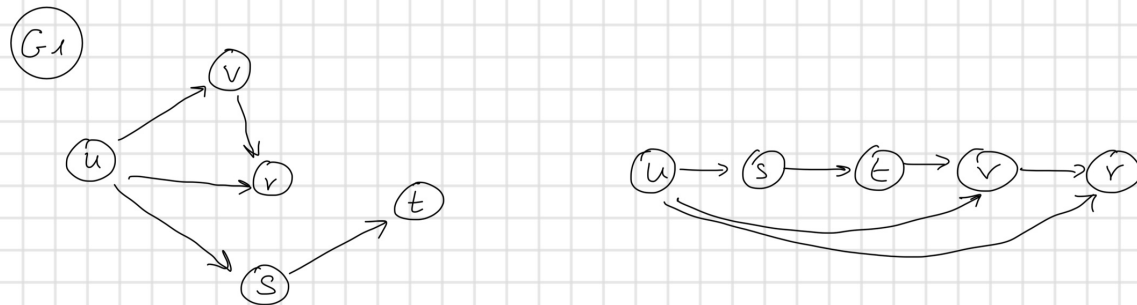
$DIST[v] = DIST[u] + 1$
 $PARENT[v] = u$
 $F. ENQUEUE(v)$

→ elaborazione dei nodi non visitati

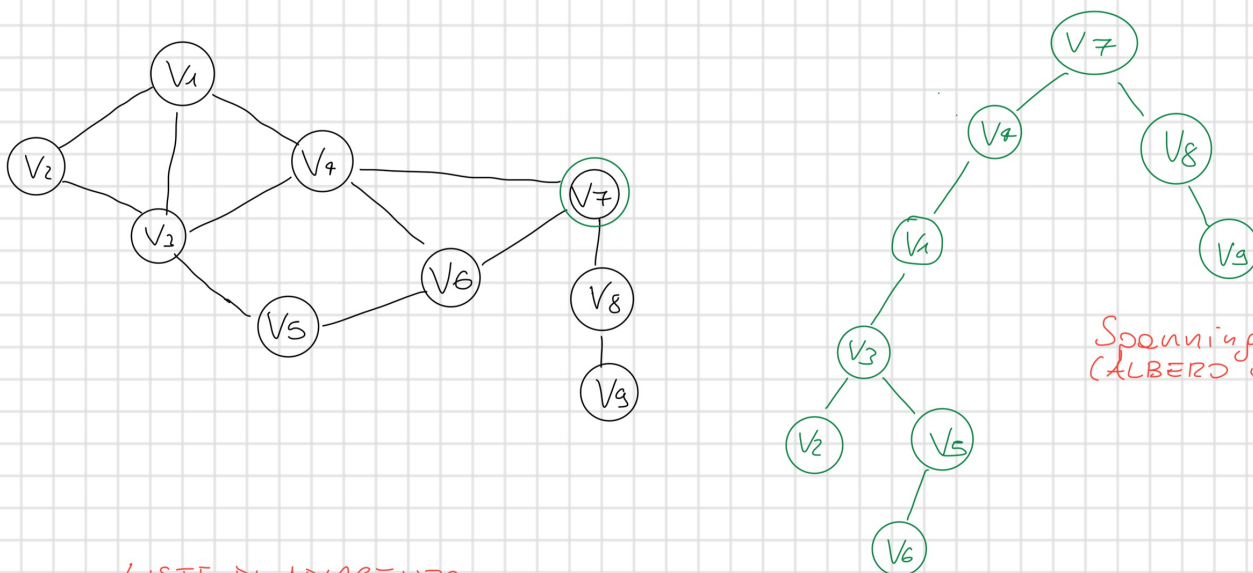
VISITA PER PROFONDITÀ (DFS depth first search)

- di solito viene utilizzato come subroutine per risolvere altri problemi, con il metodo di partire da un nodo e cercare di arrivare il più lontano possibile da esso.
- viene utilizzato per esplorare un intero grafo e non soltanto i nodi raggiungibili da una possibile sorgente.
- partendo da un nodo sorgente s se rimangono nodi non esplorati si ripete la ricerca in profondità su di essi;
- pertanto come output di questo algoritmo si ottiene una foresta, anziché un solo albero.

- **ORDINAMENTO TOPOLOGICO**, a partire da un grafo aciclico orientato.
 ↳ ordinamento lineare dei vertici del grafo (se $\exists$ un percorso da u a v , u comparirà prima nell'ordinamento)
 $u \rightarrow s \rightarrow t \rightarrow v \rightarrow r$

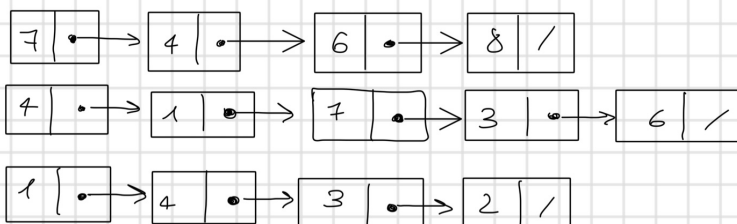


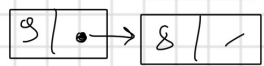
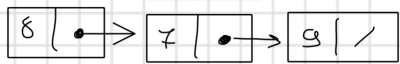
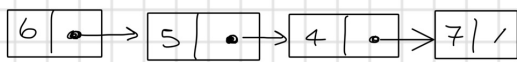
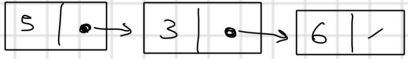
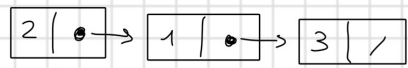
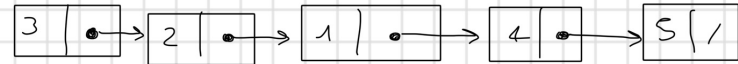
- calcolo delle **COMPONENTI (FORTEMENTE) CONNESSE** di un grafo (non) orientato.



Spanning tree
(ALBERO DI COPERTURA)

LISTE DI ADIACENZA





VIS

T	T	T	T	T	F	T	T	T
---	---	---	---	---	---	---	---	---

1 2 3 4 5 6 7 8 9

PARENT

4	3	1	7	3	5	.	7	8
---	---	---	---	---	---	---	---	---

1 2 3 4 5 6 7 8 9

(8,0)
(8,2)
(6,0)
(5,1)
(2,0)
(3,4)
(1,2)
(4,1)
(7,3)

STACK

PSEUDOCODICE

DFS(G)

for each v in v] inizializzazione
 $vis[v] = false$
 $parent[v] = null$

for each v in v] applicazione della visita fin quando ho nodi non visitati
 if ($vis[v] == false$)
 $DFS(G, v)$

DFS(G, v)

$vis[v] = true$

$stack.push(v)$

while (not $stack.empty$)

$(v, i) \leftarrow stack.top$

while ($i < len(adj[v])$ AND $adj[v, i] == true$)

$i \leftarrow i + 1$

if ($i == len(adj[v])$)

$stack.pop$

] elimino l'ultimo elemento

] ciclo necessario a leggere il primo nodo non visitato della lista di adiacenza

else

$u \leftarrow \text{adj}[v, i]$

] metto in u il primo elemento non visitato della lista di v

$\text{vis}[u] = \text{true}$

$\text{stack.top.i} \leftarrow i$

$\rightarrow$ aggiornano l'indice corrente della lista di adiacenza

$\text{parent}[u] \leftarrow v$

$\text{stack.push}(u, 1)$

$\rightarrow$ metto u nello stack